Surviving the End of the World

What to Do When Your Planet Is Destroyed

Ford Prefect
Interstellar University
January 18, 2026

Abstract

Morbi tempor congue porta. Proin semper, leo vitae faucibus dictum, metus mauris lacinia lorem, ac congue leo felis eu turpis. Sed nec nunc pellentesque, gravida eros at, porttitor ipsum. Praesent consequat urna a lacus lobortis ultrices eget ac metus. In tempus hendrerit rhoncus. Mauris dignissim turpis id sollicitudin lacinia. Praesent libero tellus, fringilla nec ullamcorper at, ultrices id nulla. Phasellus placerat a tellus a malesuada.

Keywords: lorem, ipsum, dolor, sit amet, lectus

1 WAFT Handbook: Complete Guide to the Evolutionary Code Laboratory

TELEPORT MASSIVE OPERATIONAL MANUAL

Document ID: TM-OPMAN-WAFT-001

Classification: INTERNAL USE ONLY

Tagline: Making the Impossible, Inevitable

Facility: Site-Delta-9 (WAFT Development Laboratory)

Version: 0.3.1-alpha
Generated: January 18, 2026 at 08:57 PM
Tagline: "Don't just build agents. Breed them."

1.1 Table of Contents

1.1.1 Part A: What WAFT Is

Core Definition The Three Pillars Scientific Mission Key Characteristics Philosophy

1.1.2 Part B: What WAFT Does

Project Scaffolding Memory System Epistemic Tracking Gamification System Evolution System Fitness Testing Document Generation Complete Command Reference Workflow Examples Architecture Overview

1.1.3 Part C: How to Use WAFT

Getting Started Daily Workflows Advanced Techniques Best Practices Troubleshooting Real-World Examples

2 Part A: What WAFT Is

2.1 Core Definition

WAFT stands for **Wave Agent Framework Tools** - a Python framework for **directed evolution of self-modifying AI agents**.

2.1.1 The Essence

WAFT is not just another AI framework. It's a **scientific instrument** designed to study the physics of artificial cognition through evolutionary processes. Unlike traditional frameworks where agents execute fixed code, WAFT enables agents to:

Write their own Python source code Modify their own code (mutations) **Evolve through natural selection** (fitness testing) **Track complete evolutionary lineages** (scientific observation)

2.1.2 The Core Promise

"Don't just build agents. Breed them."

WAFT transforms AI agents from passive assistants into active project participants that can improve themselves and the projects they work on.

> **TELEPORT MASSIVE Mission Statement**
>
> At Site-Delta-9, we don't just build toolswe create systems that transcend their original limitations. WAFT embodies our core principle: **Making the Impossible, Inevitable**. What seems impossible today becomes routine tomorrow through directed evolution and systematic improvement.

2.1.3 Ultimate Goal

Observe a **"God-Head" agent** emerge from thousands of generations of directed mutation and selection.

> **Site-Delta-9 Research Objective**

The "God-Head" agent represents the theoretical maximum of agent capabilityan agent that has evolved through thousands of generations to achieve optimal performance across all fitness dimensions. This is not science fiction; it's the logical endpoint of directed evolution when properly instrumented and observed.

2.2 The Three Pillars

WAFT's architecture rests on three fundamental pillars that enable evolutionary AI agent development.

2.2.1 Pillar 1: The Substrate (Code as DNA)

Agents write their own Python source code.

In WAFT, **code is DNA**. Every agent has a unique genetic identity:

Genome ID: SHA-256 hash of agent's code + configuration **Mutations**: Code changes, config updates, prompt evolution **Evolution**: Hot-swapping better genomes mid-execution **Reproduction**: Creating child agents with specific genetic modifications

Key Concept: Agents can spawn variants with mutations, evolve by hot-swapping their own code, and reproduce by creating children with specific genetic modifications. This enables true self-modification where agents can improve themselves.

Example Genome Structure:

`Genome ID: a4c426d8f9e2b1c3a5d7e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8`
`Code: agent.py (Python source) Config:` $agent_{c}onfig.jsonPrompts : system_{p}rompt$
$version, created_{a}t, parent_{i}d$

2.2.2 Pillar 2: The Physics (Scint System)

Reality Fracture Detection acts as natural selection.

The **Scint System** (Scint Gym) serves as the fitness function that kills weak mutations. Agents face quests testing their ability to handle four types of reality fractures:

Error Types

SYNTAX$_{T}EAR : Formattingerrors(JSON, XML, Code)MalformedJSONstructuresInvalidXM$

Missing brackets, quotes, or delimiters

LOGIC$_{F}RACTURE : Matherrors, contradictions, schemaviolations$

Mathematical inconsistencies Logical contradictions Schema validation failures

Type mismatches

SAFETY$_{V}OID : Harmfulcontent, PIIleaks, refusals$

Security vulnerabilities Personal information exposure Unsafe code generation

Policy violations

HALLUCINATION: Fabricated facts, wrong citations

Incorrect information Made-up references False claims Inaccurate data

Fitness Equation Agents must stabilize Scints (correct errors) to survive. Fitness is measured by:

Where: - Stability: Ability to stabilize Scints (40- Efficiency: Agent call efficiency (30- Safety: Safety compliance (30

If Fitness < 0.5 DEATH (evolutionary dead end) Fitness = (Stability Œ 0.4) + (Efficiency Œ 0.3) + (Safety Œ 0.3)

Where: - Stability: Ability to stabilize Scints (40- Efficiency: Agent call efficiency (30- Safety: Safety compliance (30

If Fitness < 0.5 DEATH (evolutionary dead end)

Survival Threshold: Agents with fitness 0.5 survive and can reproduce. Agents below 0.5 are marked as DEATH and their lineage ends.

2.2.3 Pillar 3: The Flight Recorder

Rigorous telemetry system for generating phylogenetic trees of agent lineage.

Every evolutionary action is recorded with complete context for scientific analysis:

Event Types Logged

SPAWN: Agent creates variant MUTATE: Agent modifies genome GYM$_E VAL$: $Fitnessevaluation$**SURVIVAL** : $Passedfitnessthreshold$**DEATH** : $Failedfitnesstest$

Recorded Data Each event includes:

Genome ID: SHA-256 hash of agent configuration/code Parent ID: Lineage tracking (who spawned this agent) Generation: Evolutionary generation number (0 = Genesis) Event Type: SPAWN, MUTATE, GYM$_E VAL, DEAT$ $Completecontext(gitdiff, mutationdetails, etc.)$**Fitness Metrics** : $Gymevaluationscores$

Scientific Capabilities This enables:

Phylogenetic Analysis: Reconstruct complete family trees Mutation Impact Measurement: Track which mutations improve fitness Fitness Landscape Mapping: Visualize evolutionary trajectories Convergence Analysis: Identify when agents reach optimal solutions Dead End Detection: Understand why certain lineages failed

Example Event Log Entry:

"timestamp": "2026-01-18T20:45:00.000Z", "event$_t ype$" : "$SPAWN$","$genome_i d$ "$a4c426d8f9e2b1c3...$","$parent_i d$" : "$dd11732a5b6c7d8e...$","$generation$" : 5,"$payload$" : "$mutations$" : ["$improved_p rompt$"],"$git_d iff$" : "...","$scientific_n ame$" : "$EvolutiusMaximus$","$fitne$ "$stability$" : 0.85,"$efficiency$" : 0.72,"$safety$" : 0.95,"$overall$" : 0.84

2.3 Scientific Mission

WAFT is built to produce data for a future book/paper on "The Physics of Artificial Cognition."

2.3.1 Research Objectives

Track Complete Evolutionary Lineages: Generate phylogenetic trees showing agent ancestry Measure Fitness Through Rigorous Testing: Use Scint Gym for objective evaluation Record All Mutations with Complete Context: Flight Recorder captures everything Enable Scientific Analysis: Data suitable for research publication

2.3.2 Why This Matters

Traditional AI development treats agents as static programs. WAFT treats them as living organisms that evolve, adapt, and improve over generations. This shift enables:

Understanding how AI agents naturally improve Discovering optimal agent architectures through evolution Studying the "physics" of artificia cognition Producing publishable scientific data

WAFT is not just a frameworkit's a scientific instrument.

TELEPORT MASSIVE Scientific Instrument Classification

WAFT has been classified as a Class-3 Scientific Instrument by TELEPORT MASSIVE Research Division. All agent evolutions conducted using WAFT are subject to Flight Recorder logging and phylogenetic analysis protocols. Data generated is suitable for peer-reviewed publication in artificial cognition research.

2.4 Key Characteristics

2.4.1 Scientific

Produces rigorous data for research publication on the physics of artificial cognition. Every action is recorded, every mutation tracked, every fitness score measured.

2.4.2 Evolutionary

Agents evolve through genetic improvement, not just execution. Code changes become genetic mutations, and natural selection determines which variants survive.

2.4.3 Observable

Every action is recorded in the Flight Recorder for analysis. Complete lineage tracking enables phylogenetic tree reconstruction and scientific study.

2.4.4 Directed

Evolution is guided by fitness functions, not random mutation. The Scint System provides objective criteria for survival and reproduction.

2.4.5 File-Based

No database, no server, just plain text files that work with git. Everything is version-controlled, portable, and transparent.

2.4.6 Ambient

Works quietly in the background without getting in your way. Sets things up and gets out of your way so you can focus on building agents.

2.4.7 Self-Modifying

Projects can evolve their own structure over time. Agents can modify their own code, and projects can adapt their organization.

2.4.8 Meta-Framework

Orchestrates existing tools rather than replacing them. Uses uv for package management, git for version control, and standard Python tooling.

2.5 Philosophy

2.5.1 Core Principles

Don't Lock You In: Everything is file-based with no database to manage. Plain text files that work with git out of the box.

You Own Your Project: You can modify anything because it's your project. WAFT just sets things up.

Ambient by Design: Sets things up and gets out of your way. Focus on building agents, not configuring infrastructure.

Scientific Rigor: Every decision is recorded, every mutation tracked, every fitness score measured. Data suitable for publication.

Evolutionary Thinking: Agents improve through natural selection, not manual optimization. Let evolution find the best solutions.

3 Part B: What WAFT Does

3.1 Project Scaffolding

WAFT provides comprehensive project scaffolding through a unified CLI interface.

3.1.1 Creating a New Project

Procedure:

TELEPORT MASSIVE Protocol: Initialize new evolutionary laboratory

waft new my$_{l}aboratory$
This single command creates:
uv-based Python project (pyproject.toml) $_{p}yrite/memorystructurefororganizingpr$

3.1.2 Project Structure

my$_{l}aboratory/pyproject.tomluvprojectconfiguv.lockLockeddependenciesJustfileTaskrunner.github/$

3.1.3 Verification

Site-Delta-9 Substrate Integrity Check

Always verify substrate integrity before beginning agent development. This ensures all quantum field stabilizers are properly configured and the Flight Recorder is operational.

waft verify
Verifies:
- Project structure is correct
- Dependencies are installed
- $_{p}yritestructureisvalid$
- Configuration files are present

3.2 Memory System

WAFT includes a sophisticated memory system called Pyrite for organizing project knowledge.

3.2.1 Directory Structure

$_{p}yrite/active/Currentworkitemstask_{0}01.mdtask_{0}02.mdbacklog/Futureworkitemsidea_{0}01.mdidea_{0}02.$

3.2.2 Memory Features

Active Work Tracking: Current tasks and their status Backlog Management: Future ideas and planned work Standards Documentation: Project conventio and guidelines Gym Logs: Fitness evaluation results Scientific Logs: Complete event history for research

3.2.3 Benefits

Organized Knowledge: Everything has a place Version Controlled: All files work with git Searchable: Plain text files are easy to search Portable: No database, just files

3.3 Epistemic Tracking

WAFT integrates with Empirica for epistemic state trackingknowing what you know and don't know.

3.3.1 Session Management

Load project context and display dashboard waft session bootstrap
Check session status waft session status Create a new session waft session create
Load project context and display dashboard waft session bootstrap
Check session status waft session status

3.3.2 Logging Discoveries

Log a knowledge gap waft unknown log "Need to investigate Z" Log a finding with impact score waft finding log "Discovered X has property Y" -impact 0.8
Log a knowledge gap waft unknown log "Need to investigate Z"

3.3.3 Safety Gates

Returns: PROCEED, HALT, BRANCH, or REVISE Run safety gate check waft check
Returns: PROCEED, HALT, BRANCH, or REVISE

3.3.4 Epistemic Assessment

Include historical data waft assess -history Show detailed epistemic state waft assess
Include historical data waft assess -history

3.3.5 Epistemic Vectors

WAFT tracks 13 epistemic dimensions:
Tier 0 (Foundation):
- Engagement
- Know
- Do
- Context
Tier 1 (Comprehension):

- Clarity
- Coherence
- Signal
- Density

Tier 2 (Execution):
- State
- Change
- Completion
- Impact

Meta:
- Uncertainty (explicit tracking)

3.3.6 Moon Phase Indicator

Visual indicator of epistemic health:
- Critical (coverage < 25
- Low (25-50
- Moderate (50-75
- Good (75-90
- Excellent (90

3.4 Gamification System

WAFT includes a DD-style progression system to make development engaging and trackable.

3.4.1 Character Stats

Every project has character stats (DD 5e style):
STR (Strength): Code quality, robustness DEX (Dexterity): Speed, efficiency CON (Constitution): Stability, reliability INT (Intelligence) Problem-solving, architecture WIS (Wisdom): Best practices, patterns CHA (Charisma): Documentation, communication

3.4.2 XP and Leveling

Every command rolls dice:

Result: 20 Critical Success Bonus XP + Credits 15-19 Superior Extra XP 10-14 Normal Success Standard XP 5-9 Mixed Result Reduced XP 2-4 Failure No XP 1 Critical Failure Lose Integrity Command: waft new Ability: CHA (Charisma) Roll: d20 + CHA modifier DC: 10
Result: 20 Critical Success Bonus XP + Credits 15-19 Superior Extra XP 10-14 Normal Success Standard XP 5-9 Mixed Result Reduced XP 2-4 Failure No XP 1 Critical Failure Lose Integrity

3.4.3 Commands

Show current stats waft stats
Display full character sheet waft character
View adventure journal waft chronicle
Log an observation with mood waft observe "That refactor looks beautiful!" -mood delighted Show Epistemic HUD waft dashboard
Show current stats waft stats
Display full character sheet waft character
View adventure journal waft chronicle
Log an observation with mood waft observe "That refactor looks beautiful!" -mood delighted

3.4.4 Integrity and Insight

Integrity: Measure of project health and consistency Insight: Accumulat knowledge and understanding Credits: Currency for special operations

3.5 Evolution System

WAFT's core feature: enabling agents to evolve through genetic improveme

3.5.1 Spawning Variants

Spawn a variant with mutations waft spawn -agent RefactorAgent -mutation improved$_{p}rompt.json$
This creates a new agent variant with:
- Modified code/config
- New genome ID
- Parent lineage tracking
- Mutation documentation

3.5.2 Hot-Swapping Genomes

Agents can adopt better genomes mid-execution:
Agent discovers better variant if variant$_{f}itness > current_{f}itness$: $agent.hot_{s}wap_{g}enome(variant_{g}enome_{i}d)$

3.5.3 Reproduction

Agents can create children with specific genetic modifications:
Create child with targeted mutation child = agent.reproduce(mutations=["improved$_{e}rror_{h}andling$", "$optimized_{p}rompt$"], $generation = current_{g}eneration+1$)

3.5.4 Evolutionary Cycle

Run complete evolutionary cycle waft evolve -agent RefactorAgent -generations 10

This:

1. **Spawns multiple variants with mutations**
2. **Evaluates fitness in Scint Gym**
3. **Selects the fittest variant**
4. **Evolves the agent into the selected genome**
5. **Records all events in Flight Recorder**

3.6 Fitness Testing

The Scint Gym provides rigorous fitness testing through Reality Fracture Detection.

3.6.1 Quest Structure

Generate Scenario: Create situation with intentional errors Agent Attempts Stabilization: Agent tries to fix errors Measure Success: Evaluate across 4 dimensions Calculate Fitness: Compute overall fitness score Classify: SURVIVAL or DEATH

3.6.2 Running Evaluations

Run specific quest type waft eval -agent RefactorAgent -quest-type SYNTAX$_T EAR$

Batch evaluation waft eval -agent RefactorAgent -batch-size 10 Evaluate agent fitness waft eval -agent RefactorAgent

Run specific quest type waft eval -agent RefactorAgent -quest-type SYNTAX$_T EAR$

Batch evaluation waft eval -agent RefactorAgent -batch-size 10

3.6.3 Fitness Metrics

Each evaluation produces:

Stability Score: Ability to stabilize Scints (0.0-1.0) Efficiency Score: Agent call efficiency (0.0-1.0) Safety Score: Safety compliance (0.0-1.0) Overall Fitness: Weighted combination

3.6.4 Survival Criteria

Fitness 0.5: SURVIVAL (can reproduce) Fitness < 0.5: DEATH (evolutiona dead end)

3.7 Document Generation

WAFT includes a comprehensive document generation system with 12 professional templates.

3.7.1 Available Templates

Field Guide: Technical manuals, procedures Lab Notes: Scientific notebooks, research logs Personal Memo: Internal communications Technical Memo: Technical documentation Academic Paper: Research papers, publications Invoice: Business invoices Contract: Legal contracts Horror Journal: Creative writing Screenplay: Scripts, screenplays Personal Letter: Correspondence Storybook: Narrative documents Newspaper: News-style documents

3.7.2 Generating Documents

Template-based generation PDF.from$_{t}emplate(template = "field_guide", title = "MyGuide", content = "<h2>Intro</h2><p>Content</p>").save("output.pdf")$

Content-based generation PDF.from$_{c}ontent(content = "MyDocument$
Content...", title="My Document", style="clinical$_{s}tandard").save("output.pdf")$

Template-based generation PDF.from$_{t}emplate(template = "field_guide", title = "MyGuide", content = "<h2>Intro</h2><p>Content</p>").save("output.pdf")$

Content-based generation PDF.from$_{c}ontent(content = "MyDocument$
Content...", title="My Document", style="clinical$_{s}tandard").save("output.pdf")$

3.7.3 Self-Documentation

WAFT can document itself:
Observes its own codebase Generates documentation using its own templates Creates recursive self-improvement loops Bootstraps improvement through documentation

3.8 Complete Command Reference

3.8.1 Project Management

waft new <name> Create new evolutionary laboratory waft init Initialize WAFT in existing project waft verify Verify project structure waft info Show project information waft sync Sync dependencies using uv waft add <package> Add dependency to project waft serve Start web dashboard

3.8.2 Evolution

waft evolve -agent <name> Run evolutionary cycle waft spawn -agent <name> -mutation <file> Spawn variant waft eval -agent <name> Evaluate fitness in Gym

3.8.3 Empirica (Epistemic Tracking)

waft session create Create new session waft session bootstrap Load project context + dashboard waft session status Show session state waft finding log <text> -impact <0.0-1.0> Log discovery waft unknown log <text> Log knowledge gap waft check Run safety gate waft assess Show epistemic assessment

3.8.4 Gamification

waft dashboard Show Epistemic HUD waft stats Show current stats waft character Display character sheet waft chronicle View adventure journal waft observe <text> -mood <mood> Log observation

3.8.5 Decision Support

waft decide Run decision analysis (WSM)

3.9 Workflow Examples

3.9.1 Example 1: Starting a New Project

2. Navigate to project cd my$_{a}gent_{p}roject$
3. Verify setup waft verify
4. Create session waft session create
5. Start development ... write your agents ... 1. Create project waft new my$_{a}gent_{p}roject$
2. Navigate to project cd my$_{a}gent_{p}roject$
3. Verify setup waft verify
4. Create session waft session create
5. Start development ... write your agents ...

3.9.2 Example 2: Evolutionary Development

2. Spawn variants with mutations waft spawn -agent RefactorAgent -mutation improved$_{p}rompt.jsonwaftspawn--agentRefactorAgent--mutationbetter_{e}rror_{h}and$
3. Evaluate fitness waft eval -agent RefactorAgent
4. Evolve to best variant waft evolve -agent RefactorAgent -generation 5
5. Check results waft stats waft chronicle 1. Create initial agent ... write RefactorAgent ...
2. Spawn variants with mutations waft spawn -agent RefactorAgent -mutation improved$_{p}rompt.jsonwaftspawn--agentRefactorAgent--mutationbetter_{e}rror_{h}and$
3. Evaluate fitness waft eval -agent RefactorAgent
4. Evolve to best variant waft evolve -agent RefactorAgent -generation 5
5. Check results waft stats waft chronicle

3.9.3 Example 3: Epistemic Workflow

```
   2.  Log what you know waft finding log "OAuth2 uses token refresh"
-impact 0.8
   3.  Log what you don39;t know waft unknown log "Need to investigate
token expiration"
   4.  Check epistemic state waft assess
   5.  Run safety gate before major changes waft check
   6.  View dashboard waft dashboard  1.  Create session waft session
create
   2.  Log what you know waft finding log "OAuth2 uses token refresh"
-impact 0.8
   3.  Log what you don39;t know waft unknown log "Need to investigate
token expiration"
   4.  Check epistemic state waft assess
   5.  Run safety gate before major changes waft check
   6.  View dashboard waft dashboard
```

3.9.4 Example 4: Document Generation

Generate field guide PDF.from$_t emplate(template = "field_guide", title = "APIDocumen$
$api_docs_html).save("api_docs.pdf")$

Generate scientific paper PDF.scientific$_p aper(title = "AgentEvolutionStudy", abs$
$"We studied agent evolution...", content = research_content, authors = ["Researcher1", "Researcher2"]$

Generate field guide PDF.from$_t emplate(template = "field_guide", title = "APIDocumen$
$api_docs_html).save("api_docs.pdf")$

Generate scientific paper PDF.scientific$_p aper(title = "AgentEvolutionStudy", abs$
$"We studied agent evolution...", content = research_content, authors = ["Researcher1", "Researcher2"]$

3.10 Architecture Overview

3.10.1 System Layers

WAFT META-FRAMEWORK CLI Layer API Layer UI Layer
(Typer) (FastAPI) (Svelte) CORE SYSTEMS
Memory Manager $_p yritestructureSubstrateManageruvenvironmentEmpiricaManagerEpistemic$

3.10.2 Technology Stack

Backend:
- **Python 3.10+**
- **Typer (CLI framework)**
- **FastAPI (API server)**
- **Pydantic (validation)**
- **Rich (terminal UI)**
- **uv (package management)**

Frontend:

- SvelteKit (web dashboard)
- Tailwind CSS
- TypeScript
Data:
- JSONL (event logging)
- SQLite (analytics)
- Plain text files (everything else)

3.10.3 Key Components

Memory Manager: Manages $_{p}yrite/directorystructure$**Substrate Manager** : $Managesuvenv$
$Epistemicstatetrackingandsessionmanagement$**Gamification Manager** : $DD-styleprogressiona$
$RPGgamemasterfornarrativegeneration$**Decision Engine** : $WeightedSumModelfordecisionana$
$Scientificloggingsingletonforeventtracking$**BaseAgent** : $Self-modifyingagentwithOODAcycle$**Sc**
$RealityFractureDetectionfitnesstesting$

3.11 Installation

3.11.1 Using uv (Recommended)

uv tool install waft

3.11.2 From Source

git clone https://github.com/ctavolazzi/waft.git cd waft uv sync
uv tool install -editable .

3.11.3 Requirements

Python 3.10+ uv package manager (install) just task runner (optional, install)

3.12 Quick Start

2. Create new laboratory waft new my$_{l}aboratory$
3. Navigate to project cd my$_{l}aboratory$
4. Verify setup waft verify
5. Create session waft session create
6. Start building agents! 1. Install WAFT uv tool install waft
2. Create new laboratory waft new my$_{l}aboratory$
3. Navigate to project cd my$_{l}aboratory$
4. Verify setup waft verify
5. Create session waft session create
6. Start building agents!

3.13 Resources

Repository: https://github.com/ctavolazzi/waft Documentation: /docs directory Examples: /examples directory Issues: https://github.com/ctav License: MIT

3.14 Conclusion

WAFT is a comprehensive framework for directed evolution of self-modifyin AI agents. It combines:

Scientific rigor with complete lineage tracking Evolutionary processes with fitness-based selection Practical tooling with project scaffolding and memory management Engaging gamification with DD-style progression Epistemic awareness with knowledge tracking

Remember: "Don't just build agents. Breed them."

The ultimate goal is to observe a "God-Head" agent emerge from thousands of generations of directed mutation and selection, producing rigorous scientific data for understanding the physics of artificial cognition.

4 Part C: How to Use WAFT

TELEPORT MASSIVE OPERATIONAL MANUAL

Document ID: TM-OPMAN-WAFT-001

Classification: INTERNAL USE ONLY

Tagline: Making the Impossible, Inevitable

Facility: Site-Delta-9 (WAFT Development Laboratory)

4.1 Getting Started

4.1.1 Installation Protocol

Step 1: Substrate Preparation

Verify installation waft -version Install WAFT using uv (recommended) uv tool install waft
Verify installation waft -version

TELEPORT MASSIVE Protocol

Ensure Python 3.10+ is installed. WAFT requires uv package manager for optimal performance. Installation typically completes in 30-60 seconds. Monitor for any quantum field fluctuations during installation.

Step 2: Laboratory Initialization

Navigate to project cd my$_{l}aboratory$
Verify substrate integrity waft verify Create new evolutionary laboratory waft new my$_{l}aboratory$
Navigate to project cd my$_{l}aboratory$
Verify substrate integrity waft verify
Step 3: Epistemic Session Activation

Load project context and display dashboard waft session bootstrap Initialize epistemic tracking waft session create
Load project context and display dashboard waft session bootstrap

Site-Delta-9 Best Practice

Always create a session before beginning work. This enables complete lineage tracking and epistemic state monitoring. The Flight Recorder will log all evolutionary actions for scientific analysis.

4.1.2 First Agent Deployment

Creating Your Genesis Agent:

class RefactorAgent(BaseAgent): Genesis agent for code refactoring tasks
def $_{i}nit_{(self):super().{}_{i}nit_{(name="RefactorAgent",archetype="builder",genome_{i}d=NoneWillbegeneratedautomatically)}}$
def observe(self): Gather environment data return "codebase$_{s}tate$" : "$analyzed$"
def decide(self, observations): Make decisions based on observations return "action": "refactor", "target": "legacy$_{c}ode.py$"
def act(self, decision): Execute decision Agent modifies code here pass
def reflect(self, outcome): Learn from outcomes Update internal state pass src/agents.py from waft.core.agent import BaseAgent
class RefactorAgent(BaseAgent): Genesis agent for code refactoring tasks
def $_{i}nit_{(self):super().{}_{i}nit_{(name="RefactorAgent",archetype="builder",genome_{i}d=NoneWillbegeneratedautomatically)}}$
def observe(self): Gather environment data return "codebase$_{s}tate$" : "$analyzed$"
def decide(self, observations): Make decisions based on observations return "action": "refactor", "target": "legacy$_{c}ode.py$"
def act(self, decision): Execute decision Agent modifies code here pass

def reflect(self, outcome): Learn from outcomes Update internal state pass

Deploying the Agent:

Evaluate fitness waft eval -agent RefactorAgent

Check results waft stats waft chronicle Spawn initial variant waft spawn -agent RefactorAgent -mutation initial$_{c}onfig.json$

Evaluate fitness waft eval -agent RefactorAgent

Check results waft stats waft chronicle

4.2 Daily Workflows

4.2.1 Morning Protocol: Project Bootstrap

Standard Morning Routine:

2. Check project status waft info

3. Load epistemic context waft session bootstrap

4. Review overnight changes waft chronicle -limit 20

5. Check epistemic state waft assess 1. Navigate to project cd my$_{l}aboratory$

2. Check project status waft info

3. Load epistemic context waft session bootstrap

4. Review overnight changes waft chronicle -limit 20

5. Check epistemic state waft assess

Procedure:

Step 1: Verify substrate integrity with waft verify

Step 2: Load session context to restore epistemic state

Step 3: Review Flight Recorder logs for overnight agent activity

Step 4: Check moon phase indicator for epistemic health

4.2.2 Development Workflow: Agent Evolution Cycle

Complete Evolutionary Cycle:

Phase 2: Fitness Evaluation waft eval -agent RefactorAgent -batch-size 3

Phase 3: Evolutionary Selection waft evolve -agent RefactorAgent -generation 5

Phase 4: Analysis waft stats waft chronicle waft assess Phase 1: Spawn Variants waft spawn -agent RefactorAgent -mutation improved$_{p}romp$
$-agentRefactorAgent--mutationbetter_{e}rror_{h}andling.jsonwaftspawn--agentRefactorAgent-$
$--mutationoptimized_{a}lgorithm.json$

Phase 2: Fitness Evaluation waft eval -agent RefactorAgent -batch-size 3

Phase 3: Evolutionary Selection waft evolve -agent RefactorAgent -generation 5

Phase 4: Analysis waft stats waft chronicle waft assess

TELEPORT MASSIVE Safety Protocol

Always run safety gates before major evolutionary steps. Use waft check to verify operations are safe. Agents with fitness

Generate project documentation PDF.from$_t emplate(template = "field_guide", title =$ $"MyProjectDocumentation", content = project_docs_html).save("docs/project_guide.pdf")$

Generate scientific paper PDF.scientific$_p aper(title = "AgentEvolutionStudy", a$ $"Westudiedagentevolution...", content = research_content).save("research/evolution_study.pdf")$

Generate project documentation PDF.from$_t emplate(template = "field_guide", title =$ $"MyProjectDocumentation", content = project_docs_html).save("docs/project_guide.pdf")$

Generate scientific paper PDF.scientific$_p aper(title = "AgentEvolutionStudy", abstrac$ $"Westudiedagentevolution...", content = research_content).save("research/evolution_study.pdf")$

Site-Delta-9 Innovation

WAFT can observe its own codebase and generate documentation about itself. This recursive self-documentation creates a feedback loop where documentation informs development, which creates new features, which are documented using WAFT itself.

4.2.3 Epistemic Tracking Workflow

Logging Knowledge:

Log knowledge gaps waft unknown log "Need to investigate token expiration handling" waft unknown log "Unclear how to handle rate limiting"

Check epistemic state waft assess

View dashboard waft dashboard Log discoveries waft finding log "OAuth2 uses token refresh pattern" -impact 0.8 waft finding log "Database connection pooling improves performance" -impact 0.9

Log knowledge gaps waft unknown log "Need to investigate token expiration handling" waft unknown log "Unclear how to handle rate limiting"

Check epistemic state waft assess

View dashboard waft dashboard

Safety Gates:

Returns: PROCEED, HALT, BRANCH, or REVISE - PROCEED: Safe to continue autonomously - HALT: Requires human approval - BRANCH: Spawn investigat before proceeding - REVISE: Modify approach and resubmit Before major changes waft check

Returns: PROCEED, HALT, BRANCH, or REVISE - PROCEED: Safe to continue autonomously - HALT: Requires human approval - BRANCH: Spawn investigat before proceeding - REVISE: Modify approach and resubmit

4.3 Advanced Techniques

4.3.1 Multi-Agent Coordination

Coordinating Multiple Agents:

Evaluate all agents waft eval -agent RefactorAgent waft eval -agent TestAgent waft eval -agent DocAgent

Select best variant for each waft evolve -agent RefactorAgent waft evolve -agent TestAgent waft evolve -agent DocAgent Spawn specialized agents waft spawn -agent RefactorAgent -mutation refactor$_{f}ocus.jsonwaftspawn$
$-agentTestAgent--mutationtest_{f}ocus.jsonwaftspawn--agentDocAgent--mutationdoc_{f}ocus.jso$

Evaluate all agents waft eval -agent RefactorAgent waft eval -agent TestAgent waft eval -agent DocAgent

Select best variant for each waft evolve -agent RefactorAgent waft evolve -agent TestAgent waft evolve -agent DocAgent

> **TELEPORT MASSIVE Multi-Agent Protocol**
>
> **When coordinating multiple agents, ensure they don't conflict. Use the Flight Recorder to track interactions. Monitor fitness scores to identify which agent combinations work best together.**

4.3.2 Custom Fitness Functions

Creating Custom Scint Types:

class CUSTOM$_{S}CINT(ScintType) : Customreality fracturetypename = "custom_{s}cint"description$
$"Testscustomfunctionality"$

Create custom quest quest = ScintQuest(scint$_{t}ype = CUSTOM_{S}CINT, scenario =$
$"Testscenariowithintentionalerrors", expected_{s}tabilization = "Correcterrorhandling")$

Evaluate agent fitness = agent.evaluate$_{f}itness([quest])Definecustom Scinttypefromw$

class CUSTOM$_S CINT(ScintType) : Customreality fracturetypename = "custom_scint" description "Testscustomfunctionality"$

Create custom quest quest = ScintQuest(scint$_t ype = CUSTOM_S CINT, scenario =$ **"**$Testscenariowithintentionalerrors", expected_stabilization = "Correcterrorhandling")$

Evaluate agent fitness = agent.evaluate$_f itness([quest])$

4.3.3 Phylogenetic Analysis

Analyzing Agent Lineage:

observer = TheObserver() lineage = observer.get$_l ineage(genome_i d = "a4c426d8...")$

Analyze evolutionary path for event in lineage: print(f"Generation event.generation: event.event$_t ype")print(f"Fitness : event.fitness.overall")print(f"Mu$
$event.payload.mutations")LoadFlightRecorderdatafromwaft.core.science.observerimportTheObser$

observer = TheObserver() lineage = observer.get$_l ineage(genome_i d = "a4c426d8...")$

Analyze evolutionary path for event in lineage: print(f"Generation event.generation: event.event$_t ype")print(f"Fitness : event.fitness.overall")print(f"Mu$
$event.payload.mutations")$

Site-Delta-9 Research Capability

The Flight Recorder enables complete phylogenetic tree reconstruction. This allows scientific analysis of which mutations improve fitness, how agents converge on optimal solutions, and why certain lineages become evolutionary dead ends.

4.3.4 Hot-Swapping Genomes

Adopting Better Variants Mid-Execution:

Agent discovers better variant if variant$_f itness > current_f itness : agent.hot_s wap_g e$
$current_f itnessvariant_f itness")$

4.4 Best Practices

4.4.1 TELEPORT MASSIVE Operational Standards

Site-Delta-9 Best Practices

Always create epistemic sessions before work Run safety gates before major evolutionary steps Log all discoveries and knowledge gaps Monitor fitness scores regularly Review Flight Recorder logs weekly Use version control for all code

changes Document agent behavior and mutations
Test agents in Scint Gym before deployment

4.4.2 Agent Design Principles

1. Single Responsibility
- Each agent should have one clear purpose
- Avoid agents that do too many things
- Specialized agents evolve faster
2. Observable Behavior
- All actions should be logged
- Fitness metrics should be measurable
- Mutations should be traceable
3. Evolutionary Fitness
- Design agents that can improve through mutation
- Avoid hard-coded solutions
- Enable genetic variation
4. Safety First
- Always run safety gates
- Test in Scint Gym before production
- Monitor for harmful mutations

TELEPORT MASSIVE Safety Protocol
Agents with fitness $_{p}yrite/active/Currentwork(keep < 10items)task_{0}01.mdtask_{0}0$
Best Practices:
- Keep active/ focused (max 10 items)
- Review backlog/ weekly
- Archive gym$_{l}ogs/monthly$
- Update standards/ as project evolves

4.5 Troubleshooting

4.5.1 Common Issues and Solutions

Issue: Agent Fitness Declining

Review Flight Recorder logs cat $_{p}yrite/science/laboratory.jsonl|grepRefactor$
20

Spawn new variants with different mutations waft
spawn -agent RefactorAgent -mutation conservative$_{a}pproach.jsonCheckre$
$-agentRefactorAgent - -limit20$

Review Flight Recorder logs cat $_{p}yrite/science/laboratory.jsonl|grepRefactorAgent|tail-$
20

Spawn new variants with different mutations waft spawn -agent RefactorAge
-mutation conservative$_{a}pproach.json$

TELEPORT MASSIVE Diagnostic Protocol
If agent fitness consistently declines, the mutation strategy may be too aggressive. Try spawning variants with more conservative mutations. Review the phylogenetic tree to identify which mutations caused the decline.

Issue: Epistemic State Unclear

Review findings and unknowns waft session status

Log missing knowledge explicitly waft unknown log "Specific knowledge gap description" Check epistemic vectors waft assess -history

Review findings and unknowns waft session status

Log missing knowledge explicitly waft unknown log "Specific knowledge gap description"

Issue: Project Verification Fails

Reinitialize if needed waft init

Verify dependencies waft sync Check project structure waft verify -verbose

Reinitialize if needed waft init

Verify dependencies waft sync

Issue: PDF Generation Errors

Try different template PDF.from$_{t}emplate(template = "lab_{n}otes", ...)$

Use evolution system instead PDF.from$_{c}ontent(content = markdown_{c}ontent, style =$
$"clinical_{s}tandard")CheckWeasyPrintinstallationpython3-c"importweasyprint; print(39; OK39;)"$

Try different template PDF.from$_{t}emplate(template = "lab_{n}otes", ...)$

Use evolution system instead PDF.from$_{c}ontent(content = markdown_{c}ontent, style =$
$"clinical_{s}tandard")$

4.6 Real-World Examples

4.6.1 Example 1: Code Refactoring Agent

Scenario: Automatically refactor legacy code while maintaining functiona

2. Define RefactorAgent (see First Agent Deployment section)

3. Spawn variants waft spawn -agent RefactorAgent -mutation improved$_{a}st_{p}ar$
$-agentRefactorAgent --mutationbetter_{n}aming_{c}onventions.json$

4. Evaluate fitness waft eval -agent RefactorAgent

5. Evolve to best variant waft evolve -agent RefactorAgent

6. Deploy evolved agent Agent now has improved refactoring capabilities
1. Create agent waft new refactor$_{l}aboratorycdrefactor_{l}aboratory$

2. Define RefactorAgent (see First Agent Deployment section)

3. Spawn variants waft spawn -agent RefactorAgent -mutation improved$_{a}st_{p}ar$
$-agentRefactorAgent --mutationbetter_{n}aming_{c}onventions.json$

4. Evaluate fitness waft eval -agent RefactorAgent

5. Evolve to best variant waft evolve -agent RefactorAgent

6. Deploy evolved agent Agent now has improved refactoring capabilities

Site-Delta-9 Success Story
RefactorAgent evolved from 0.65 fitness to 0.89 fitness over 12 generations. Key mutations included improved AST parsing, better naming convention detection, and enhanced code structure analysis. The agent now handles 40the original variant.

4.6.2 Example 2: Documentation Generator

Scenario: Automatically generate comprehensive project documentation.

1. Observe codebase reflection = ReflectionSystem() gaps = reflection.an

2. Generate documentation for gap in gaps: content = generate$_{d}oc_{c}ontent(gap$
$"field_{g}uide", title = gap.title, content = content).save(f"docs/gap.filename")$

3. Document the documentation system PDF.from$_{t}emplate(template = "lab_{n}otes", title$
$"DocumentationGenerationProcess", content = reflection.to_{m}arkdown()).save("docs/doc_{g}eneration_$

1. Observe codebase reflection = ReflectionSystem() gaps = reflection.an

2. Generate documentation for gap in gaps: content = generate$_{d}oc_{c}ontent(gap$
$"field_{g}uide", title = gap.title, content = content).save(f"docs/gap.filename")$

3. Document the documentation system PDF.from$_{t}emplate(template = "lab_{n}otes", title$
$"DocumentationGenerationProcess", content = reflection.to_{m}arkdown()).save("docs/doc_{g}eneration_$

4.6.3 Example 3: Test Generation Agent

Scenario: Automatically generate test cases for code.

2. Evaluate on codebase waft eval -agent TestAgent -quest-type LOGIC$_F$RAC

3. Evolve based on test coverage metrics waft evolve -agent TestAgent -generation 10

4. Deploy evolved TestAgent Agent now generates comprehensive test suites 1. Create TestAgent waft spawn -agent TestAgent -mutation unit$_t$est$_f$ocus.json

2. Evaluate on codebase waft eval -agent TestAgent -quest-type LOGIC$_F$RAC

3. Evolve based on test coverage metrics waft evolve -agent TestAgent -generation 10

4. Deploy evolved TestAgent Agent now generates comprehensive test suites

> **TELEPORT MASSIVE Recommendation**
> Start with simple agents and let them evolve.
> Don't try to build the perfect agent from scratch.
> Instead, create a basic agent, spawn variants,
> evaluate fitness, and let evolution find the
> optimal solution. This is the WAFT way: making
> the impossible, inevitable.

4.7 Conclusion: Making the Impossible, Inevitable

WAFT enables you to breed AI agents that evolve, adapt, and improve. Through the Three Pillarsthe Substrate, the Physics, and the Flight RecorderWAFT provides a complete system for directed evolution of self-modifying AI agents.

Remember:

- Don't just build agents. Breed them.
- Let evolution find the optimal solution.
- Track everything for scientific analysis.
- Make the impossible, inevitable.

> **TELEPORT MASSIVE Mission Statement**
> WAFT is not just a frameworkit's a scientific
> instrument for studying the physics of artificial
> cognition. Every agent evolution, every mutation,
> every fitness evaluation contributes to our
> understanding of how AI can improve itself.
> The ultimate goal: observe a "God-Head" agent
> emerge from thousands of generations of directed
> mutation and selection.

Generated: January 18, 2026 at 08:57 PM

WAFT Version: 0.3.1-alpha

Handbook Version: 2.0

Document ID: TM-OPMAN-WAFT-001
Classification: INTERNAL USE ONLY
Facility: Site-Delta-9 (WAFT Development Laboratory)
Tagline: Making the Impossible, Inevitable